CHARUSAT®
अमृतं तु विद्या
CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY

FACULTY OF TECHNOLOGY AND ENGINEERING
DEVANG PATEL INSTITUTE OF ADVANCE TECHNOLOGY AND RESEARCH

DEPARTMENT OF INFORMATION TECHNOLOGY

A.Y 2026 – 27 Semester – 3

LAB MANUAL

CSUC201 Fundamentals of Data Structures and Algorithms

DEPSTAR

Practical-9.1

Github Link: https://github.com/rudraaaajoshi/FDSA_BATCH_A/tree/main/Practical-9

Difficulty

Understanding the difference between DFS and BFS was slightly challenging.

Error

Wrongly choosing or marking a visited building can give an incorrect traversal order.

Solution

Used DFS with recursion/stack to go deep first and BFS with a queue to visit buildings level by level.

Outcome

Successfully implemented both traversals and displayed their different visiting orders.

Code(Github) Link:

https://github.com/rudraaaajoshi/FDSA_BATCH_A/blob/main/Practical-9/pr-9.1.cpp

Output:

```
DFS: 0 1 3 4 2
BFS: 0 1 2 3 4
Process returned 0 (0x0)   execution time : 1.059 s
Press any key to continue.
```

Key Question:

Problem-1

Both strategies visit every reachable node exactly once. If the goal is simply to check whether a specific building exists in the network, which strategy would find it sooner if the building is very close to the start? What if it is very far and deep? Explain your reasoning without tracing through any specific example.

Answer:

If the building is very close to the start, BFS will usually find it sooner because BFS checks nearby nodes first, level by level.
If the building is very far and deep, DFS may find it sooner because DFS explores deeply along one path before backtracking. The exact time depends on the network structure.

Q- Your DFS implementation likely uses recursion. What real-world constraint on the computer could cause your recursive DFS to fail on a very large city map, and how would you restructure your solution to avoid that failure.

Answer:

Recursive DFS can fail because a very large/deep city map may cause stack overflow when too many recursive function calls are stored in memory.
To avoid this, use iterative DFS with an explicit stack (stack) instead of recursion. This lets the program manage the traversal stack itself.